

Sudoku Game

Opis użytych narzędzi i konfiguracji

June 15, 2026

Contents

1	Cel dokumentu	2
2	CMake	2
3	Doxygen	2
4	clang-uml	3
5	LaTeX	4
6	Wersje narzędzi użyte podczas przygotowania dokumentacji	4

1 Cel dokumentu

Dokument opisuje narzędzia użyte przy przygotowaniu projektu Sudoku Game: kompilację programu, generowanie dokumentacji programistycznej oraz generowanie diagramu klas. Informacje można wykorzystać do odtworzenia środowiska i ponownego wygenerowania artefaktów projektu.

2 CMake

CMake służy do konfiguracji i budowania programu C++. Projekt korzysta z pliku `CMakeLists.txt` w katalogu głównym.

Najważniejsze ustawienia:

- minimalna wersja CMake: 3.16,
- język projektu: C++,
- standard języka: C++17,
- wynikowy program: `Sudoku_Game`.

Typowy build lokalny:

```
cmake -S . -B build
cmake --build build
```

Build pliku `.exe` dla Windows z poziomu Linuksa:

```
cmake -S . -B build/windows-x64 \
  -DCMAKE_SYSTEM_NAME=Windows \
  -DCMAKE_CXX_COMPILER=x86_64-w64-mingw32-g++ \
  -DCMAKE_BUILD_TYPE=Release

cmake --build build/windows-x64 --parallel
```

Wariant z linkowaniem statycznym bibliotek GCC i standardowej biblioteki C++:

```
cmake -S . -B build/windows-x64-static \
  -DCMAKE_SYSTEM_NAME=Windows \
  -DCMAKE_CXX_COMPILER=x86_64-w64-mingw32-g++ \
  -DCMAKE_BUILD_TYPE=Release \
  "-DCMAKE_EXE_LINKER_FLAGS=-static -static-libgcc -static-libstdc++"

cmake --build build/windows-x64-static --parallel
```

3 Doxygen

Doxygen został użyty do wygenerowania dokumentacji programistycznej na podstawie komentarzy w plikach źródłowych C++.

Konfiguracja znajduje się w pliku `Doxyfile`. Najważniejsze parametry:

Parametr	Znaczenie
----------	-----------

<code>PROJECT_NAME = "Sudoku"</code>	Nazwa projektu w dokumentacji.
<code>INPUT = core main.cpp</code>	Pliki i katalogi analizowane przez Doxygen.
<code>RECURSIVE = YES</code>	Rekurencyjne przeglądanie katalogów wejściowych.
<code>EXTRACT_ALL = YES</code>	Do dokumentacji trafiają również elementy bez pełnych komentarzy.
<code>GENERATE_HTML = YES</code>	Generowanie dokumentacji HTML.
<code>HTML_OUTPUT = html</code>	Katalog wynikowy HTML.
<code>GENERATE_LATEX = YES</code>	Generowanie dokumentacji LaTeX.
<code>LATEX_OUTPUT = latex</code>	Katalog wynikowy LaTeX.
<code>HAVE_DOT = YES</code>	Użycie Graphviz do diagramów zależności.
<code>CALL_GRAPH = YES</code>	Generowanie grafów wywołań funkcji.
<code>CALLER_GRAPH = YES</code>	Generowanie grafów funkcji wywołujących.
<code>DOT_IMAGE_FORMAT = png</code>	Format grafik Graphviz.

Polecenie generujące dokumentację:

```
doxygen Doxyfile
```

Dokumentacja HTML jest dostępna w `html/index.html`. Dokumentacja LaTeX jest generowana w katalogu `latex`. Plik PDF dokumentacji programistycznej można zbudować poleceniem:

```
make -C latex
```

Wynikowy plik to `latex/refman.pdf`.

4 clang-uml

`clang-uml` został użyty do wygenerowania diagramu klas z kodu C++. Konfiguracja znajduje się w pliku `.clang-uml`.

Aktualna konfiguracja:

```

compilation_database_dir: build
output_directory: uml

diagrams:
  sudoku_class_diagram:
    type: class
    glob:
      - main.cpp
      - core/*.cpp
      - core/*.h
    include_system_headers: false

```

Znaczenie parametrów:

- `compilation_database_dir` wskazuje katalog z plikiem `compile_commands.json`;
- `output_directory` wskazuje katalog wynikowy diagramów;
- `type: class` oznacza diagram klas;
- `glob` określa analizowane pliki projektu;

- `include_system_headers: false` pomija klasy z nagłówków systemowych.

Przed uruchomieniem `clang-uml` warto upewnić się, że istnieje aktualna baza kompilacji:

```
cmake -S . -B build -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
```

Diagram generuje się poleceniem:

```
clang-uml
```

Wyniki są zapisywane w katalogu `uml`. Najważniejsze pliki:

- `uml/sudoku_class_diagram.puml`,
- `uml/sudoku_class_diagram.svg`,
- `uml/sudoku_class_diagram.png`.

5 LaTeX

LaTeX został użyty do przygotowania dokumentacji przeznaczonej dla zwykłego użytkownika oraz dokumentu opisującego użyte narzędzia. Źródła znajdują się w katalogu `docs`.

Pliki źródłowe:

- `docs/user_manual.tex` – instrukcja użytkownika,
- `docs/tools_used.tex` – opis narzędzi i konfiguracji.

PDF-y można zbudować poleceniem:

```
make -C docs
```

Wyniki:

- `docs/user_manual.pdf`,
- `docs/tools_used.pdf`.

6 Wersje narzędzi użyte podczas przygotowania dokumentacji

W aktualnym środowisku użyto:

- Doxygen 1.16.1,
- `clang-uml` 0.6.2,
- CMake 4.3.3,
- `x86_64-w64-mingw32-g++` 16.1.0.